

# MCP (Model Context Protocol) — das Grundkonzept im Überblick

---

Quellen: [video-summary-sQBinJA\\_zxU.md](#), [claude-oekosystem-ueberblick.md](#), [video-summary-B\\_OqkMRFonM.md](#), [video-summary-AL391nkWGlc.md](#), [video-summary-1guudCDr0H4.md](#)

MCP wird in mehreren Videos am Rand erwähnt, aber nur in zweien konkret vorgeführt bzw. aus erster Hand erklärt (Home Assistant als Praxisbeispiel; ein Anthropic-Mitentwickler-Interview zu Entstehung und Governance). Dieser Artikel bündelt das allgemeine Konzept mit beidem (siehe [notes-audit-report.md](#)).

## Was MCP ist

---

Model Context Protocol ist die offene Schnittstelle, die KI-Absichten in konkrete API-Aufrufe übersetzt — Analogie: MCP ist der Autoschlüssel, ein [Skill](#) ist die Fahrstunde (sQBinJA\_zxU). B\_OqkMRFonM beschreibt es als die offene Schnittstelle hinter den Connectors (Gmail, Google Calendar, Notion, Slack) — vergleichbar mit einem einheitlichen USB-Anschluss für KI-Tools: verschiedene Anbieter/Dienste stellen ihre Funktionen über dasselbe Protokoll bereit, statt dass jedes Tool eine eigene Integration bräuchte.

## Entstehung bei Anthropic und Abgrenzung zu API/CLI (1guudCDr0H4, Interview mit Mitentwickler David Soria Parra)

---

Interner Ausgangspunkt war die Beobachtung, dass ein zentrales Developer-Tooling-Team immer langsamer wächst als der Rest der Firma — die einzige Skalierungsoption sei, Mitarbeitenden Werkzeuge zu geben, mit denen sie sich selbst Integrationen bauen können. Ursprünglicher interner Arbeitstitel: "**Claude Connect**". Weil es gleichzeitig mehrere Clients gab (Claude Desktop, IDEs wie Zed, VS Code), entstand das klassische N-Clients-x-M-Integrationen-Problem, woraus die Idee eines gemeinsamen Protokolls wurde.

Einordnung gegenüber den zwei naheliegenden Alternativen:

- **Gegenüber klassischen APIs:** API-Endpunkte sind für programmatischen Zugriff gedacht und oft zu granular für Modelle. MCP sei "nicht viel mehr als eine API mit einem semantischen Layer" für Authentifizierung, automatisches Nachfragen und Human-in-the-Loop-Bestätigungen.
- **Gegenüber CLI-Zugriff:** Für einzelne Entwickler/kleine Firmen reicht CLI. Größere Organisationen, die keine Binärdatei auf jeden Laptop verteilen, sondern einen zentralen Server mit Policy-Enforcement/Governance-Layer wollen, sind mit einem Protokoll wie MCP besser bedient.

## Governance: wie MCP zum offenen Industriestandard wurde

---

Zwei bewusste Entscheidungen von Anfang an, damit MCP "wirklich" offen bleibt:

1. **MIT-Lizenz, bewusst ohne Contributor License Agreement (CLA)** — laut David Soria Parra, damit Anthropic die Lizenz später nicht ändern kann, ohne alle jemals beteiligten Mitwirkenden

fragen zu müssen (CLA-Detail nur aus dieser Primärquelle, nicht unabhängig gegengeprüft).

2. **Übertragung an eine neutrale Stiftung:** Anthropic übergab MCP im Dezember 2025 an die neu gegründete **Agentic AI Foundation (AAIF)** unter dem Dach der **Linux Foundation** — acht Platin-Mitglieder (AWS, Anthropic, Block, Bloomberg, Cloudflare, Google, Microsoft, OpenAI) plus Technical Steering Committee. Zweck: Firmen sollen auf MCP aufbauen können, ohne dass eine einzelne Firma es zurückziehen könnte.

Übertragbare Lektion für eigene interne Standards/Tools, die firmenübergreifend Vertrauen brauchen: frühe Lizenzwahl ohne Rücknahmemöglichkeit plus Übergabe an eine neutrale Instanz.

## Praxisbeispiel: MCP für Home Assistant (AL391nkWGlc)

Home Assistant hat eine eigene API, über die normalerweise nur Apps zugreifen. MCP setzt sich dazwischen und öffnet diese API strukturiert für KI-Modelle — nicht nur lesend, sondern auch schreibend/erstellend.

**Zwei Varianten mit sehr unterschiedlichem Umfang:**

| Eingebauter MCP-Server            | Community-Projekt "ha-mcp"                                                        |
|-----------------------------------|-----------------------------------------------------------------------------------|
| Entität für "Assist" freigegebene | alle                                                                              |
| Automationen/Szenen               | erstellen und bearbeiten                                                          |
| Dashboards                        | bauen                                                                             |
| Delegierter Zugriff auf Traces    | Automationen anhand von Traces/Logs debuggen                                      |
| Weiteres                          | Helper/Areas/Zones/Labels/Groups, Backups, Add-ons, HACS, Geräte-/Entity-Registry |

**Faustregel:** Der eingebaute Server reicht für Sprachsteuerung bereits freigegebener Geräte. Für aktives Bauen, Konfigurieren oder Debuggen braucht es die Community-Lösung.

**Einrichtung:** Installation über HA App-Store/HACS, inklusive Tool Security Policies, Allow-/Deny-Listen und automatischem Backup vor Änderungen. Für Fernzugriff ohne Portfreigabe: ein Webhook-Proxy (Nabu Casa), über den auch lokale KI-Clients (z. B. LM Studio) zugreifen können.

**Was das in der Praxis ermöglicht** (Beispiele aus dem Video): Claude erkennt, dass ein vorhandener Regensensor ein kumulativer Zähler ist, und legt selbstständig einen passenden Utility-Meter-Helper an; eine Kinderzimmer-Lichtautomation wird über die Automations-Trace analysiert und der Bug per Fix-Vorschlag behoben — beides wäre über den eingebauten Server (kein Trace-Zugriff, kein Helper-Erstellen) nicht möglich gewesen.

## Praktische Einordnung

Der generelle Lehrsatz aus dem Beispiel: MCP selbst ist nur das Protokoll/die Leitung. Wie viel ein Agent damit wirklich tun kann, hängt vollständig davon ab, was der jeweilige MCP-Server tatsächlich freigibt — ein "dünner" offizieller Server und ein "dicker" Community-Server können für dieselbe Anwendung völlig unterschiedliche Fähigkeiten bedeuten. Das lohnt sich vor der Einrichtung eines neuen MCP-Servers für ein anderes Tool zu prüfen, statt automatisch von voller Kontrolle auszugehen.

---

## Kernbotschaft

---

MCP ist die Übersetzungsschicht zwischen KI-Absicht und konkreten API-Aufrufen — technisch vergleichbar mit einem einheitlichen Anschluss, über den unterschiedliche Dienste ihre Funktionen bereitstellen. Wie viel Handlungsspielraum ein Agent dadurch bekommt, entscheidet aber nicht das Protokoll selbst, sondern der Funktionsumfang des jeweiligen MCP-Servers — offizielle Server sind oft bewusst eingeschränkt, Community-Server oft deutlich mächtiger.

## Themen-Tags

---

MCP, Model Context Protocol, Connectors, Home Assistant, Smart Home, Agentic Coding

## Zu prüfen

---

Offene Verifikationspunkte zum Home-Assistant-Beispiel (Feature-Vergleich, Versionsnummern, Kostenmodell des Webhook-Proxy) sind bereits in [video-summary-AL391nkWGlc.md](#) erfasst.

Quelle: mcp-ueberblick.md